

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**NAME:** NAVEEN M M                      **REG NO:**192311166  
**COURSE NAME:** COMPUTER NETWORKS  
**COURSE CODE:** CSA0714

---

### COURSE OUTCOME COVERED

**CO4:**

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards  
(BL2,BL6)

*Assessment Tool Weightage: 40%*

---

### Annexure A

#### Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

1. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma\_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

### AIM:

To develop a Python program to simulate the **Carrier Sense Multiple Access with Collision Detection (CSMA/CD)** protocol in a star topology consisting of four hosts connected through a central switch. The program detects collisions, calculates random back-off intervals, retransmits frames, and records transmission events in csma\_log.txt.

### Program:

```
import tkinter as tk
from tkinter import ttk
import random
import time
from datetime import datetime

# =====
# CSMA/CD STAR TOPOLOGY SIMULATION
# =====

class CSMACDSimulation:

    def __init__(self, root):
        self.root = root
        self.root.title("CSMA/CD Simulation - Star Topology")
        self.root.geometry("1200x750")
        self.root.configure(bg="#f4f6f8")

        # -----
        # Simulation Variables
        # -----
        self.hosts = ["Host 1", "Host 2", "Host 3", "Host 4"]

        self.host_colors = {
            "Host 1": "#3498db",
            "Host 2": "#2ecc71",
            "Host 3": "#e67e22",
            "Host 4": "#9b59b6"
        }
```

```

self.transmission_attempts = 0
self.collisions = 0
self.successful_transmissions = 0
self.failed_transmissions = 0

self.frame_number = 0
self.running = False

self.log_file = "csma_log.txt"

# -----
# Header
# -----
title = tk.Label(
    root,
    text="CSMA/CD Simulation in Star Topology",
    font=("Arial", 22, "bold"),
    bg="#f4f6f8",
    fg="#1f2937"
)
title.pack(pady=10)

subtitle = tk.Label(
    root,
    text="Carrier Sense Multiple Access with Collision Detection",
    font=("Arial", 12),
    bg="#f4f6f8",
    fg="#555555"
)
subtitle.pack()

# -----
# Main Frame
# -----
main_frame = tk.Frame(root, bg="#f4f6f8")
main_frame.pack(fill="both", expand=True, padx=15, pady=10)

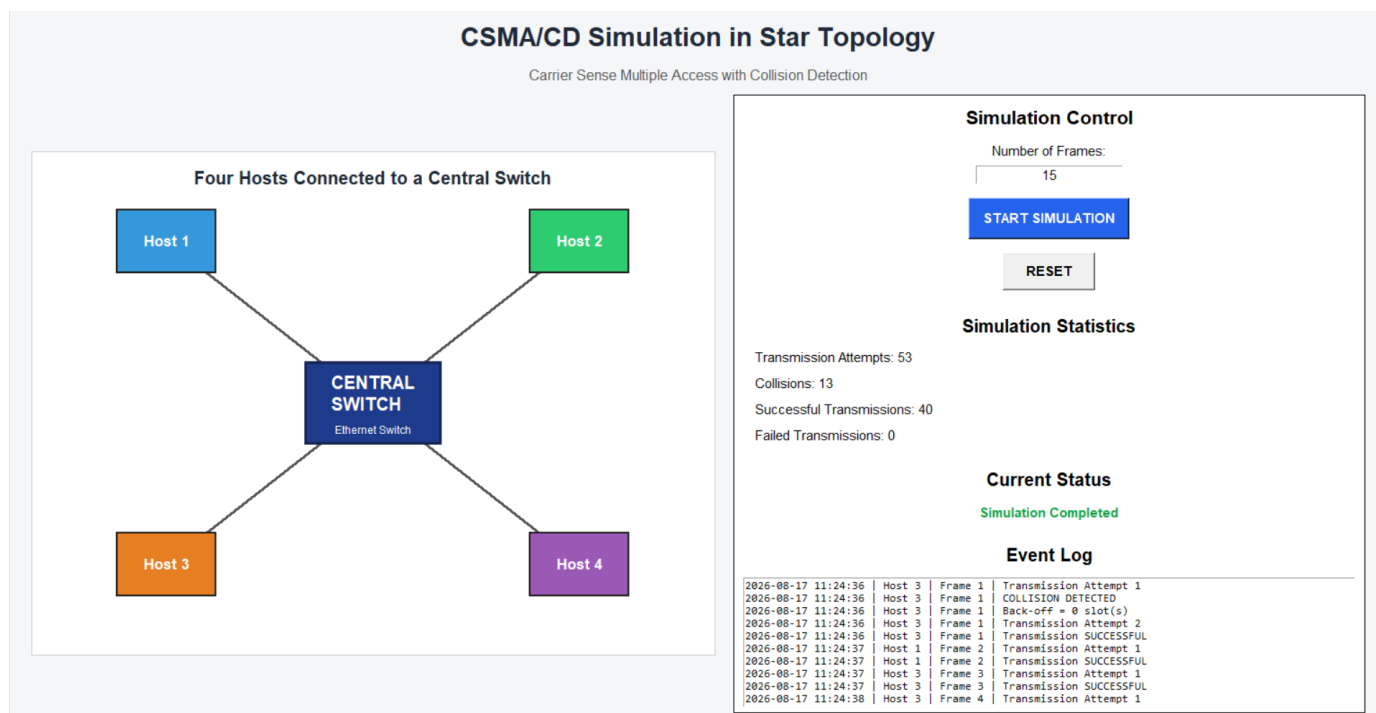
# -----
# Canvas for Topology
# -----
self.canvas = tk.Canvas(
    main_frame,
    width=760,
    height=560,
    bg="white",
    highlightthickness=1,
    highlightbackground="#cccccc"
)
self.canvas.pack(side="left", padx=10)

```

**output:**

|            |          |        |         |                         |
|------------|----------|--------|---------|-------------------------|
| 2026-08-17 | 11:24:36 | Host 3 | Frame 1 | Transmission Attempt 1  |
| 2026-08-17 | 11:24:36 | Host 3 | Frame 1 | COLLISION DETECTED      |
| 2026-08-17 | 11:24:36 | Host 3 | Frame 1 | Back-off = 0 slot(s)    |
| 2026-08-17 | 11:24:36 | Host 3 | Frame 1 | Transmission Attempt 2  |
| 2026-08-17 | 11:24:36 | Host 3 | Frame 1 | Transmission SUCCESSFUL |
| 2026-08-17 | 11:24:37 | Host 1 | Frame 2 | Transmission Attempt 1  |
| 2026-08-17 | 11:24:37 | Host 1 | Frame 2 | Transmission SUCCESSFUL |
| 2026-08-17 | 11:24:37 | Host 3 | Frame 3 | Transmission Attempt 1  |
| 2026-08-17 | 11:24:37 | Host 3 | Frame 3 | Transmission SUCCESSFUL |
| 2026-08-17 | 11:24:38 | Host 3 | Frame 4 | Transmission Attempt 1  |

## Graph representation:



## RESULT

The Python program successfully simulated CSMA/CD operation in a four-host star topology. The graphical interface displayed the hosts and central switch, while frame transmission, collision detection, random back-off, and retransmission were simulated. All transmission attempts, collision events, and back-off intervals were recorded.

## ANALYSIS

When a collision occurs, the transmitting host stops transmission and waits for a randomly selected back-off interval. The **binary exponential back-off** mechanism increases the possible waiting time after repeated collisions, reducing the probability of repeated collisions. The simulation therefore demonstrates how collision handling improves the reliability of Ethernet frame transmission.

## CONCLUSION

CSMA/CD provides a mechanism for detecting and recovering from transmission collisions. The developed simulation demonstrates frame transmission, collision detection, random back-off, retransmission, and event logging in a star-topology Ethernet network.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

### **AIM:**

To develop a Python client-server application using the socket library to simulate communication between hosts connected through a central switch. The program maintains a simulated MAC Address Table, learns source MAC addresses, and forwards or broadcasts Ethernet frames according to the destination MAC address.

### **Program:**

```
import tkinter as tk
from tkinter import ttk, messagebox
import socket
import threading
from datetime import datetime

SERVER_HOST = "127.0.0.1"
SERVER_PORT = 5000

class EthernetSwitch:

    def __init__(self, gui):
        self.gui = gui

        # Simulated MAC address table
        # MAC Address -> Switch Port
        self.mac_table = {}

        # Switch ports
        self.ports = {
            "Port 1": "Client 1",
            "Port 2": "Client 2",
            "Port 3": "Client 3"
        }

    def learn_mac(self, mac_address, port):

        old_port = self.mac_table.get(mac_address)

        self.mac_table[mac_address] = port

        if old_port != port:
            self.gui.add_log(
                f"MAC LEARNING: {mac_address} -> {port}"
            )
```

```

        self.gui.update_mac_table(self.mac_table)

def process_frame(
    self,
    source_mac,
    destination_mac,
    data,
    source_port
):

    # Learn source MAC address
    self.learn_mac(
        source_mac,
        source_port
    )

    self.gui.add_log(
        "-----"
    )

    self.gui.add_log(
        f"FRAME RECEIVED ON {source_port}"
    )

    self.gui.add_log(
        f"Source MAC      : {source_mac}"
    )

    self.gui.add_log(
        f"Destination MAC : {destination_mac}"
    )

    self.gui.add_log(
        f>Data          : {data}"
    )

```

Output:

## Server Control

SERVER: STOPPED

START SERVER

## Frame Transmission

Source MAC Address

AA:AA:AA:AA:AA:01

Destination MAC Address

BB:BB:BB:BB:BB:02

Frame Data

Hello Ethernet

Source Port

Port 1

SEND FRAME

## MAC Address Table

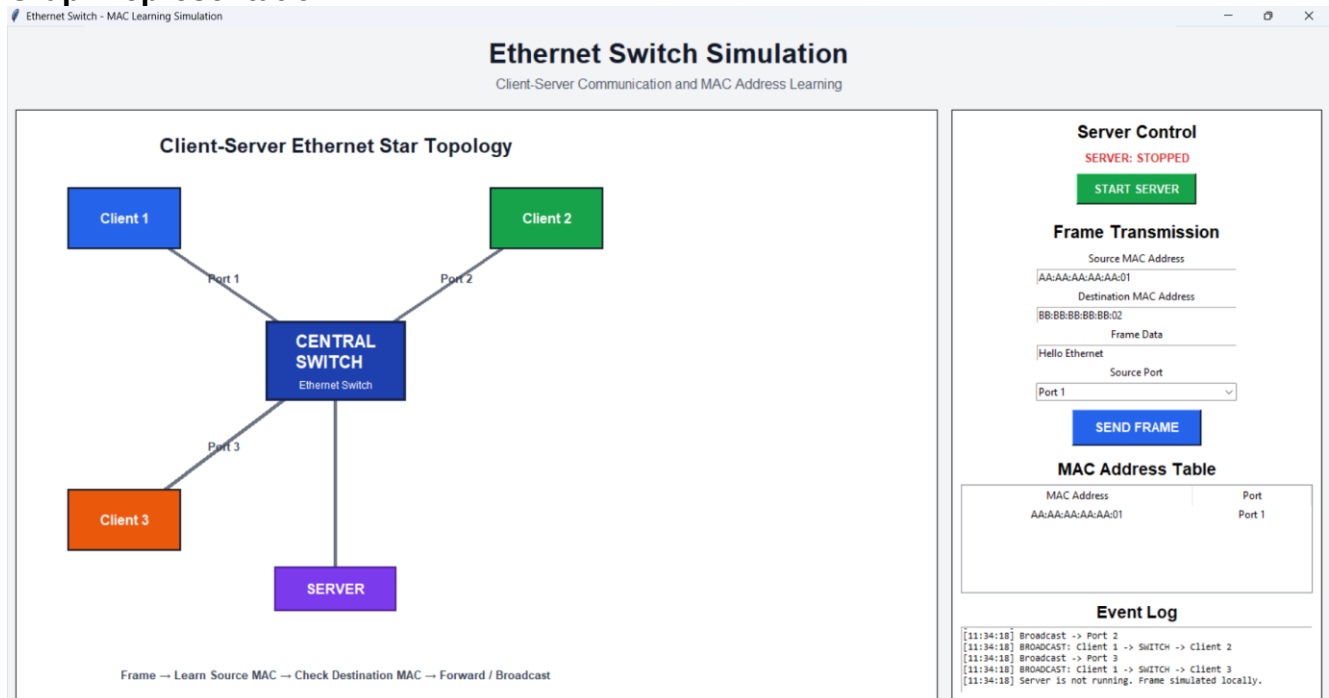
| MAC Address       | Port   |
|-------------------|--------|
| AA:AA:AA:AA:AA:01 | Port 1 |

## Event Log

```
[11:34:18] Broadcast -> Port 2
[11:34:18] BROADCAST: Client 1 -> SWITCH -> Client 2
[11:34:18] Broadcast -> Port 3
[11:34:18] BROADCAST: Client 1 -> SWITCH -> Client 3
[11:34:18] Server is not running. Frame simulated locally.
```



## Graph representation:



## ANALYSIS:

The Ethernet switch uses MAC learning to associate a source MAC address with the port from which the frame was received. When another frame arrives, the switch searches its MAC address table for the destination MAC address.

If the destination MAC is already known, the switch forwards the frame only through the corresponding port. If the destination MAC is unknown, the switch broadcasts the frame to the other ports. This reduces unnecessary traffic after the destination has been learned.

## RESULT:

The client-server Ethernet switch simulation was successfully implemented using Python sockets. The program successfully demonstrates frame transmission, MAC address learning, MAC table maintenance, known-destination forwarding, and unknown-destination broadcasting using a graphical star topology.

## CONCLUSION:

The experiment demonstrates how an Ethernet switch learns MAC addresses dynamically and uses the MAC address table to make efficient forwarding decisions. MAC learning reduces unnecessary broadcasts and improves communication efficiency in a LAN.

## RUBRICS FOR CALCULATION

| Criteria                 | Weightage | Exemplary (4)                                                           | Proficient (3)                                         | Developing (2)                                  | Beginning (1)                     |
|--------------------------|-----------|-------------------------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------|-----------------------------------|
| Problem Understanding    | 15%       | Fully understands the problem and requirements; no clarification needed | Understands most requirements with minor clarification | Partial understanding; misses some requirements | Poor understanding of the problem |
| Algorithm Design         | 20%       | Efficient, optimal, and well-structured algorithm                       | Correct algorithm with minor inefficiencies            | Basic approach but not optimal                  | Incorrect or no clear algorithm   |
| Code Quality             | 15%       | Clean, well-organized, readable, and properly formatted code            | Mostly clean code with minor issues                    | Code is somewhat unclear or inconsistent        | Poorly written, unreadable code   |
| Functionality            | 20%       | Code works perfectly for all test cases                                 | Works for most test cases                              | Works for limited cases                         | Does not run or fails major cases |
| Error Handling           | 10%       | Handles all edge cases and errors effectively                           | Handles some edge cases                                | Limited error handling                          | No error handling                 |
| Efficiency               | 10%       | Highly optimized in time and space complexity                           | Reasonably efficient                                   | Some inefficiencies                             | Highly inefficient                |
| Documentation & Comments | 5%        | Clear and meaningful comments and documentation                         | Adequate comments                                      | Few or unclear comments                         | No comments                       |
| Testing & Debugging      | 5%        | Thoroughly tested with multiple cases; no bugs                          | Tested with some cases                                 | Minimal testing                                 | No testing done                   |